# Gujarat Disaster Management Platform: Member-wise Sprint Tasks

## How to read this

- Task IDs (S1-01 … S3-18) match the Task Board workbook; the owner changes made in this revision are listed under “Changes in this revision”, and the workbook needs the same changes. **Owner** means accountable for the deliverable. **Support** means you help the owner and take your input from them.
- **Split of work**: when a task has a support member, the owner’s part and each support member’s part are written as separate lines (Owner (name), Support (name)). The support member also has a short entry for that task in their own section.
- **Already in repo** names code that exists today. Extend it; do not rebuild it.
- **Needs from** is a prerequisite from another member. If it is not there by the day shown, tell Siddh the same day.
- **Gives to** is who is waiting for your output.Team and assignment

Each member now has a name. The Task Board workbook still uses the earlier codes, so they are listed here for cross-reference. Dhruv (QA and test lead) and Dhruvil (ML) are two different people.

| Member | Group | Role in this plan |
|---|---|---|
| Siddh | Leader | Tech lead and integration owner; owns CI/CD and releases; holds and manages every repo and hosting secret |
| Zalak | Frontend & backend | Citizen and public backend, duplicate-grouping endpoint, LLM adapter (GenAI), AI-versus-human analytics, final report |
| Nisarg | Frontend & backend | Dispatch and field-team backend, public API and Gujarat geofence, team recommendations and team-request decisions |
| Shlok | Frontend & backend | Payments, signal promotion API, shared dispatch-transition refactor; full-stack |
| Rudra | ML | Ingestion and external alerts, AI service skeleton, prompt-injection and adversarial post sets |
| Neeti | ML (also GenAI) | AI lead: dataset design, text model, evaluation, /triage integration, LLM-versus-model decision |
| Dhruvil | ML | Image model, duplicate text-similarity model and labelled pairs, in-process model loading, photo-analyse path |
| Janvi | Frontend & backend | Frontend lead for citizen and field screens |
| Tanishq | Frontend & backend (also GenAI) | Frontend for the public site and the dispatcher workspace |

| Member | Group | Role in this plan |
|---|---|---|
| Dhruv | Frontend & backend | QA and test lead: test harness, smoke and scenario tests, role matrix, defect triage (CI/CD now sits with Siddh) |

### How people were assigned

- **Secrets**: Siddh holds and manages every repo and hosting secret (JWT_SECRET, database, Redis and RabbitMQ credentials, INGEST_TOKEN, payment gateway keys, LLM keys, and the TOTP secret of the smoke-test account). Any step that creates, rotates or enters a secret is Siddh's. Everyone else refers to secrets by environment variable name only and uses dummy values locally.
- **ML work**: Rudra, Neeti and Dhruvil do every ML-related task: text and image datasets and their labelling, models, training and evaluation, embeddings, the duplicate text-similarity model, the /triage service and loading the model inside the backend, AI pre-labels for ingested signals, and adversarial or prompt-injection test sets. No one else is assigned these. Neeti signs off AI changes made by Dhruvil, Rudra and Zalak before merge.
- **GenAI work**: Siddh, Zalak, Tanishq and Neeti can take GenAI tasks when needed: the LLM adapter, running the LLM in evaluations, English summaries of Hindi or Gujarati text, and LLM-generated synthetic text (flagged as synthetic, training split only, never in the test set). In this plan Zalak builds the LLM adapter, Neeti decides whether it stays on, and Siddh handles keys.

## Already built: reuse, do not rebuild

| Area | What exists | Where |
|---|---|---|
| Staff login | Password plus TOTP MFA, JWT, role and permissions read from the DB on every request, lockout, audit | backend/app/auth.py, security.py, routers/auth.py, cli.py |
| Incident handling | Verify, reject, needs-info, priority, closure, reopen; rule-based duplicate candidates with human link, merge or dismiss | backend/app/routers/dm/incidents.py, services/incidents.py |
| Dispatch | Idempotent creation, acknowledge, reject, progress, complete, fail; row-locked resource reservation; auto-escalation every 60 s | routers/dm/dispatches.py, services/dispatch.py, services/resources.py, main.py |
| AI boundary | Provider contract and guardrails, advisory recommendations stored separately, human decision recorded, 'unavailable' path | services/ai_provider.py, routers/dm/ai.py, frontend dm/incidents/[id]/ai-panel.tsx |
| Dispatcher UI | 12 workspace pages, BFF proxy, httpOnly session cookie | frontend/src/app/dm/*, app/api/session, app/api/dm/[...path], lib/api.ts |

| Area | What exists | Where |
|---|---|---|
| Home map | Leaflet map, OSM tiles, Nominatim search, About modal | frontend/src/app/page.tsx, components/dm/LiveOperationsLayer.tsx |
| Legacy portal | Mock data in localStorage (Delhi, Odisha, Kerala), mock triage and duplicates, mock role dashboards | services/resqStorage.ts, context/ResQContext.tsx, aiTriage.ts, duplicateDetection.ts, components/role-views/* |
| Database | 21 tables plus staff_permissions, PostGIS indexes, hash-chained audit log, versioned migration runner | backend/db/migrations/01 and 02, app/migrate.py |
| Citizen module (stand-in) | Merged 16 to 18 Oct: SQLite store, POST and GET /api/citizen/reports that trust an X-User-Id header, /api/public/incidents, alerts and shelters from a demo seed, a citizen dashboard page, citizenApi.ts, a test file outside tests/. Replace it with the real API; do not extend it | backend/main.py, database.py, models.py, test_citizen_api.py, frontend/src/app/citizen, services/citizenApi.ts |
| Money tables | donations, donation_campaigns, aid tables, DM oversight and campaign pages (no gateway) | schema 01, routers/dm/aid.py, dm/campaigns |
| Image model | ConvNeXt-tiny, 7 classes; README reports 0.839 accuracy and 0.815 macro-F1; weights exist only on a Drive link | ai-workers/model/* |
| Tests and CI | 59 backend test functions; ci.yml (path filtering, gitleaks, lint, ruff, bandit, pip-audit, a postgis/postgis:16-3.4 job with migrations, audit-hash check and pytest, Docker build matrix with Trivy); cd.yml publishing images to GHCR; Dockerfiles, compose, Render and Vercel config | backend/tests, .github/workflows/ci.yml and cd.yml, docker-compose*.yml, render.yaml |
| Docs | Accurate DM module guide; ER diagram, FR/NFR/API PDF, user stories workbook | docs/, documents/ |

**Not built yet (this plan builds it):** citizen registration and a report API on the real database with verified tokens (the merged citizen module is a SQLite stand-in), the public API, field-team role, payment gateway, external ingestion, Gujarat data, text AI model, team recommendations.

## Sprint goals

- **Sprint 1: real core loop.** A citizen registers and reports, a dispatcher verifies, and the verified incident shows on the public Gujarat map, all on the real backend. The AI dataset and baselines, the ingestion framework and the payments foundation start in parallel. Day 3: that loop demoed. Day 5: tested on staging.
- **Sprint 2: run the response and integrate AI.** Team recommendation and confirmed assignment, the field-team flow, the text AI model evaluated and integrated, duplicate scoring, 10-minute ingestion, donation sandbox. Day 3: dispatch to field loop. Day 5: AI integrated and tested.
- **Sprint 3: harden and release.** End-to-end scenario, security review, polish, AI error analysis, deployment, docs, rehearsals. Day 3: full scenario on staging. Day 5: tagged release candidate.

## Sprint-wise work of each member

### Sprint 1: real core loop

| Member | Owns | Supports | Work in this sprint | Waiting on |
|---|---|---|---|---|
| Siddh | S1-01, S1-02, S1-18, S1-19 | S1-07, S1-09, S1-13, S1-16, S1-17 | Rotate secrets and enter them in Render, Vercel, Neon and GitHub; contract template and sign-off; migrations 03 and 04, ingestion user and Gujarat seed; CI additions (ran and skipped guard, smoke step) and staging deploy; source reachability probe for Rudra; compose entry for the AI service; Day 3 and Day 5 gates | Dhruv (README draft, variable-name list, test guard, smoke script), Nisarg and Shlok (seed parts), each owner (contract section) |
| Zalak | S1-03, S1-04 | S1-02, S1-09 | Citizen register and login API; citizen report API with the Gujarat check and my-reports timeline; retire the SQLite stand-in; fix citizen-incident backend defects | Siddh (migration 04, CI), Nisarg (geo.py), Dhruv (DB fixtures) |
| Nisarg | S1-07, S1-11 | S1-02, S1-04, S1-09 | Public verified-incidents API and geo.py (replaces the SQLite public routes); team-recommendation rules and query; Gujarat agencies and teams seed; run the dispatch flow on citizen incidents | Siddh (migration 04, district list) |
| Shlok | S1-16 | S1-02, S1-09 | Payment gateway interface, MockGateway, campaigns API, create-order; campaigns and demo-incident seed; fix /dm route defects | Siddh (migration 04, contract, gateway keys if a test account is opened), Zalak (incident column list) |

| Member | Owns | Supports | Work in this sprint | Waiting on |
|---|---|---|---|---|
| Rudra | S1-13, S1-15 | S1-10 | AI service skeleton (/triage, /health) on the rules baseline; ingestion framework, USGS adapter, fallback fixtures, source-access memo; collect citizen-style posts; label about 200 texts | Neeti (rules.py, guideline), Siddh (tables, Render reachability result) |
| Neeti | S1-10, S1-12 | S1-13 | Dataset design and guideline, baselines and evaluation harness, rules port; label about 200 texts | Dhruvil and Rudra (texts), Siddh (contract) |
| Dhruvil | S1-14 | S1-10, S1-12, S1-13 | Image model weights and class mapping; collect and clean texts, label about 200; harness helpers; Dockerfile and /triage tests | Neeti (guideline), Siddh (where the model file lives), Rudra (skeleton) |
| Janvi | S1-05, S1-06 | S1-08 | Auth UI and citizen report and My reports on the real backend; move the merged citizen module off the SQLite API | Zalak (auth and report API), Nisarg (error codes), Tanishq (map constants) |
| Tanishq | S1-08, S1-09 | S1-05, S1-06 | Public Gujarat map, About and How-to pages; dispatcher console on real data; retire the mock | Nisarg (public JSON), Zalak (report API), Siddh (district list) |
| Dhruv | S1-17 | S1-01, S1-03, S1-18, S1-19 | DB test harness and fixtures, ran and skipped guard, smoke script, seed reset; README draft and variable-name list; smoke on staging; run the Day 3 checklist | Siddh (migrations, smoke-account secret, CI wiring), Zalak and Nisarg (endpoints) |

**Sprint 2: run the response and integrate AI**

| Member | Owns | Supports | Work in this sprint | Waiting on |
|---|---|---|---|---|
| Siddh | S2-15, S2-16 | S2-01, S2-02, S2-06, S2-08, S2-09, S2-10, S2-12 | Staging redeploy on every merge, post-deploy smoke, ingestion cron workflow; AI image and model artifact in CI and CD; review the dispatch refactor, team-request decisions and signal promotion; enter LLM key, ingest token and gateway secrets; Day 3 and Day 5 gates | Dhruv (extended smoke), Rudra (ingest endpoint), Neeti (model artifact) |

| Member | Owns | Supports | Work in this sprint | Waiting on |
|---|---|---|---|---|
| Zalak | S2-08, S2-13 | S2-07, S2-14 | LLM adapter (GenAI, off by default), citizen information response, abuse limits; duplicate endpoint using Dhruvil's similarity module; tests | Neeti (contract), Dhruvil (similarity module), Janvi (UI) |
| Nisarg | S2-01, S2-02 | S2-03, S2-14 | Field role and endpoints; team-recommendation endpoint and team-request decisions | Shlok (refactor, Day 1), Siddh (migration 04), Dhruv (fixtures) |
| Shlok | S2-10, S2-12 | S2-01 | Shared dispatch-transition refactor (Day 1); signal promotion API; donation verify, webhook, receipts and donation screens | Rudra (signals rows), Janvi (components), Siddh (secrets, review) |
| Rudra | S2-09 | S2-06, S2-10, S2-11 | CAP and SACHET adapter, social or replay source, 10-minute scheduler, public alerts API; ten prompt-injection posts | Neeti (/triage), Siddh (cron, signals table) |
| Neeti | S2-05, S2-06 | S2-07, S2-08, S2-09 | Train and select the text model; integrate /triage, priority rules, guardrail tests; sign off similarity features and the LLM adapter | Dhruvil (embedding runs), Rudra (injection posts), frozen Sprint 1 data |
| Dhruvil | S2-07 | S2-05, S2-06 | At least 100 labelled duplicate pairs, text-similarity module and tuning; InProcessProvider and EARTHQUAKE test; embedding runs | Neeti (contract, sign-off), Zalak (endpoint), Siddh (model location) |
| Janvi | S2-04, S2-11 | S2-12, S2-13 | Field-team dashboards, alerts map layer, donation and information-response UI | Nisarg (field endpoints), Rudra (alerts API), Shlok |
| Tanishq | S2-03 | S2-04, S2-06, S2-10 | Dispatcher recommendation, assign and request screens, AI panel, signals queue | Nisarg (recommendation endpoints), Shlok (signals API), Neeti |
| Dhruv | S2-14 | S2-01, S2-12, S2-15, S2-16 | Tests for new routers; extend smoke to the field flow and a donation; field-user fixtures; run the gate checklist | Zalak, Nisarg and Shlok routers, Siddh (staging) |

**Sprint 3: harden and release**

| Member | Owns | Supports | Work in this sprint | Waiting on |
|---|---|---|---|---|
| Siddh | S3-02, S3-12, S3-15, S3-18 | S3-16, S3-17 | Security and RBAC review and scans; production-like deployment, final secrets, migrations and rollback tags; rehearsals; release tag | All routers stable at Day 1 |
| Zalak | S3-06, S3-16 | S3-01, S3-02, S3-07, S3-15, S3-17 | AI-versus-human analytics endpoint; final report assembly; run the LLM on the low-confidence subset; scenario steps, role-leak checks, bug fixing | Neeti (confidence bands), Tanishq (chart), every member (sections) |
| Nisarg | S3-13, S3-14 | S3-01, S3-04, S3-15, S3-17 | EXPLAIN and indexes, tune team recommendations; safety-guidance source check; field and public bug fixing | Siddh (staging), Shlok (load script) |
| Shlok | S3-11 | S3-02, S3-12, S3-13, S3-15, S3-17 | Payment failure paths, receipts, tamper tests; backup and seed scripts; load-test script and run | Janvi (UI states), Siddh (staging) |
| Rudra | S3-09, S3-10 | S3-15, S3-16 | 24-hour ingestion soak and health panel; source finalisation and sanitisation | Siddh (host), Neeti (/triage) |
| Neeti | S3-05, S3-07 | S3-06, S3-15, S3-16, S3-17 | AI error analysis, thresholds, final model card; LLM-versus-model decision | Dhruvil (error tables, model side), Zalak (LLM runs), rehearsal decisions |
| Dhruvil | S3-08 | S3-05, S3-07, S3-15, S3-16 | Photo-analyse path with image input to /triage; error tables; model-side predictions on the subset | Neeti, Tanishq (UI) |
| Janvi | S3-03 | S3-11, S3-15 to S3-17 | Polish, accessibility, mobile, payment UI states, frontend defects | Tanishq |
| Tanishq | S3-04 | S3-03, S3-06, S3-08, S3-15 to S3-17 | Final About, How-to and safety content, polish, charts, photo UI, dispatcher and public defects | Nisarg (guidance check), Zalak (chart data) |
| Dhruv | S3-01, S3-17 | S3-02, S3-12, S3-15 | Automated cyclone scenario; defect triage; role-matrix script; clean-deploy check; demo script draft | Zalak and Nisarg steps, Siddh (staging) |

## Siddh: Tech lead, integration owner and CI/CD owner (Leader)

Owns the database changes, API contracts, the merge gate, CI/CD, the security review and the release. Reviews every pull request. CODEOWNERS currently names one person; add the module owners. Holds and manages every repo and hosting secret; no one else is given secret values.

### Sprint 1

#### S1-01 Secrets rotation, repo hygiene, README, API contract (owner; Dhruv supports)

- Already in repo: .env.example, gitleaks in CI, docs/disaster-management.md (accurate), a README that still describes the old 10-week plan.
- Do: (1) Rotate JWT_SECRET and every database, Redis and RabbitMQ credential that was in the shared ZIP, then update Render, Vercel and Neon. (2) Make sure no .env, venv or node_modules is ever shared. (3) Get the README opening rewritten for this project: Gujarat only, what is real, what is simulated. (4) Own the contract doc: path, auth, request, response and error codes for every new endpoint (public, citizen, field, signals, team recommendations, payments, ingest), using the existing error shape {error:{code,message,details}}.
- **Owner (Siddh)** rotates JWT_SECRET and every database, Redis and RabbitMQ credential, enters the new values in Render, Vercel, Neon and the GitHub Actions secrets, makes sure no .env, venv or node_modules is ever shared, sets up the contract template and the shared error shape, reviews the README opening, merges the contract and collects each owner's sign-off. Secret values stay with Siddh only.
- **Support (Dhruv)** drafts the README opening (Gujarat only, what is real, what is simulated) and gives Siddh a checklist of every environment variable name each service needs (names only, never values). Siddh confirms gitleaks and /health/ready are green after the rotation.
- **Each endpoint owner** (Zalak, Nisarg, Shlok, Rudra, Neeti) drafts their own section of the contract on Siddh's template by Day 1 and signs it off by Day 2.
- Needs from: Dhruv's README draft and variable-name checklist; each owner's contract section.
- Gives to: everyone; frontend and backend both build against the contract.
- Done when: gitleaks is green, the README is accurate, and each owner has signed off their section.

#### S1-02 Migrations 03 and 04 plus Gujarat seed (owner; Nisarg, Shlok and Zalak support)

- Already in repo: the migration runner (app/migrate.py: ordered files, tracked in schema_migrations, each file in one transaction); schema 01 and 02; seed_data.sql with generic national agencies, three scenarios and a bootstrap admin whose password hash is only a placeholder.
- Do: Migration 03 only adds enum values (EARTHQUAKE to scenario_type, FIELD_TEAM to role_code), because Postgres cannot use a new enum value in the same transaction that adds it. Migration 04: add source, source_ref, is_simulated and district to incidents; create external_alerts, ingest_runs, source_signals, field_team_members, dispatch_progress and team_requests; add gateway order and payment columns to donations; insert the FIELD_TEAM role; add earthquake to the SCENARIOS_SUPPORTED config and to safety content. Gujarat seed: Gujarat agencies (verify names), at least 10 teams as TEAM resources with Gujarat coordinates and capability JSON, campaigns, about 20 demo incidents across districts, and a system user for

ingestion. Also add earthquake to Scenario in app/common.py and SUPPORTED in services/ai_provider.py.

- **Owner (Siddh)** writes migration 03 (enum values only) and migration 04 (new incident columns, new tables, gateway columns, FIELD_TEAM role, earthquake config), creates the ingestion system user, merges the seed parts below into one Gujarat seed, adds earthquake to Scenario in app/common.py and SUPPORTED in services/ai_provider.py, sets the real bootstrap admin credential outside git, and proves fresh and upgraded databases both migrate.
- **Support (Nisarg)** writes the Gujarat agencies and at least 10 teams as seed rows with coordinates and capability keys, and agrees the keys with Siddh by Day 1.
- **Support (Shlok)** writes the seed rows for the donation campaigns and about 20 demo incidents across districts once the migration 04 columns are agreed.
- **Support (Zalak)** lists the incident columns and enum values the report API needs and reviews migration 04.
- Needs from: Nisarg's agencies and teams by Day 1; Shlok's campaigns and demo incidents by Day 3; Zalak's column list. Rudra and Neeti give their column needs through the contract doc. The CI job in S1-18 runs migrations on a fresh and an upgraded database.
- Gives to: Zalak, Nisarg, Shlok, Rudra (everything in Sprint 1 depends on these tables).
- Done when: fresh and upgraded databases both migrate and the existing 59 tests still pass.

**S1-07 Public API (support, owner Nisarg)**

- Do: review the SQL for the status filter, the verified_by requirement and coordinate rounding. Provide the Gujarat boundary source (an open GeoJSON, or a bounding box if none is found) and the district list.
- Needs from: Nisarg's draft endpoint by Day 2.

**S1-09 Dispatcher console on real data (support, owner Tanishq)**

- Do: review and merge the backend fixes that Shlok (/dm routes) and Zalak (citizen incident code) make for defects found when the /dm console runs on citizen-created incidents. Keep tests green.
- Needs from: Tanishq's defect list; Zalak's report API merged.

**S1-13 AI service skeleton (support, owner Rudra)**

- Do: confirm the /triage contract; add the AI service to docker-compose and check it is in the CI and CD image builds; set AI_TRIAGE_URL in .env.example.
- Needs from: Rudra's service and Dhruvil's Dockerfile.

**S1-16 Payments foundation (support, owner Shlok)**

- Do: hold and enter the secrets. If a free test-mode gateway account (Razorpay or Stripe) can be opened, open it, keep the keys and enter them in the hosting consoles; set the MockGateway signing secret on staging. Shlok never receives real keys.
- Needs from: Shlok's finding on whether a test-mode account is possible.

**S1-17 DB test harness and smoke script (support, owner Dhruv)**

- Do: create the staff account for the smoke script with the CLI, keep its TOTP secret and store it as a CI and staging secret. Tell Dhruv only the variable name. Wire Dhruv's ran-and-skipped guard and the smoke step into the CI workflow (see S1-18).
- Needs from: Dhruv's guard and smoke script design (the variable name it reads).

**S1-18 CI pipeline and staging deploy (owner; Dhruv supports)**

- Already in repo: ci.yml (path filtering, gitleaks, frontend lint and build, ruff, bandit, pip-audit, a postgis/postgis:16-3.4 job that runs migrations, checks the audit hash trigger and runs pytest against TEST_DATABASE_URL, Docker build matrix with Trivy); cd.yml, which pushes images to GHCR; render.yaml (free plan, image from GHCR, autoDeploy off, health at /health); frontend vercel.json (deployments off by default).
- Do: (1) CI: keep the DB job; add Dhruv's ran-and-skipped print and the step that fails when no DB test ran; add the smoke step against compose; make sure the root-level test_citizen_api.py is either run or removed with the SQLite stand-in, because CI only runs tests/. (2) Staging: create the Neon database with PostGIS; generate and enter DATABASE_URL, JWT_SECRET and INGEST_TOKEN in Render and Vercel; leave AI_TRIAGE_URL empty at first; deploy the backend image on Render and the frontend on Vercel; check /health/ready; write the cold-start note and staging URLs in the README. (3) Run each ingestion source once from Render and tell Rudra the reachability result by Day 3.
- **Owner (Siddh)** does all three items and owns every workflow, Docker, Render and Vercel setting.
- **Support (Dhruv)** writes the ran-and-skipped guard and the smoke script (S1-17), runs the smoke script against staging when it is up and reports the result, and gives Siddh the names-only variable checklist.
- Needs from: Dhruv's guard and smoke script; the S1-01 rotation done first.
- Gives to: everyone (CI and staging); Rudra (reachability result).
- Done when: the staging URL serves the public map and login, CI fails when the smoke step fails, and the executed-test count is visible in the CI log.

**S1-19 Integration gate (owner; Dhruv supports)**

- Do: Day 3, run the demo checklist on staging or compose: register, report, verify, see it on the public map. Day 5, merge to main only after CI is green and the smoke script passes.
- **Owner (Siddh)** decides the merge and holds the gate.
- **Support (Dhruv)** runs the demo checklist and the smoke script and reports pass or fail with the failing step.
- Needs from: every owner's PR green by Day 4 evening; Dhruv's smoke script.
- Done when: main runs via compose and the demo script passes end to end.

## Sprint 2

**S2-16 Staging redeploy and ingestion cron (owner; Dhruv supports)**

- Already in repo: cd.yml publishes images; render.yaml has autoDeploy off.
- Do: redeploy staging on each merge to main; run the smoke script after every deploy; add a GitHub Actions or outside cron workflow that calls /internal/ingest/run about every 10 minutes

with INGEST_TOKEN read from a secret by name; document running the scheduler on the demo laptop, because the free Render service sleeps.

- **Owner (Siddh)** builds the redeploy step, the post-deploy smoke step and the cron workflow, stores the ingest token as a GitHub Actions secret and host variable, and provides the staging webhook URL for S2-12.
- **Support (Dhruv)** extends the smoke script to the field flow and a donation, and checks that the staging webhook URL answers.
- Needs from: Rudra's endpoint and token; Dhruv's extended smoke script.
- Done when: the smoke test is green on staging and ingestion runs about every 10 minutes while the service is awake.

**S2-15 Integration gate (owner; Dhruv supports)**

- Do: same routine as S1-19 for Sprint 2, plus update the demo script with the field and AI steps. Dhruv runs the checklist; Siddh decides the merge.

**S2-01 and S2-02 (support, owner Nisarg)**

- Do: review the shared dispatch-transition refactor (Shlok) and the team-request decision endpoints (Nisarg): permissions, audit chain and idempotency. These are high-risk changes, so Tanishq or Zalak is the second reviewer.

**S2-06 AI integration (support, owner Neeti)**

- Do: make sure the AI service image builds and publishes in CI and CD, that the compose entry and AI_TRIAGE_URL work with the model artifact, and decide where the model artifact is stored (not in git). Review Dhruvil's InProcessProvider.
- Needs from: Neeti's model artifact and final contract.

**S2-08, S2-09, S2-10 and S2-12 (support)**

- S2-08 (owner Zalak): decide where provider keys live (hosting environment variables, never git), enter the key in staging and keep LLM_ENABLED off by default.
- S2-09 (owner Rudra): add INGEST_TOKEN and the other environment values to the host and to the GitHub Actions secrets so the cron workflow can refer to them by name.
- S2-10 (owner Shlok): review the promote transaction, the audit row and the negative test that a promoted signal is not public before verification.
- S2-12 (owner Shlok): review the signature-verification code and enter the gateway and webhook secrets in staging.

## Sprint 3

**S3-02 Security, RBAC and privacy review (owner; Dhruv, Zalak and Shlok support)**

- Do: run a matrix of roles (public, citizen, field, dispatcher, admin) against every route group with the expected status code. Check /public/* and /me/* for personal data. Confirm the new BFF proxies reject cross-site writes like the existing one. Confirm that the SQLite stand-in routes and the X-User-Id header trust are gone. Fix or log every finding.

- **Owner (Siddh)** reviews the matrix results, checks /public/* and /me/* for personal data, confirms the BFF proxies reject cross-site writes and the stand-in routes are gone, runs the scans and the dependency audit in CI (and decides which of them should block a merge), and fixes or logs every finding.
- **Support (Dhruv)** builds the role-matrix script and runs it; Zalak, Nisarg and Shlok give the expected status codes for their route groups.
- **Support (Zalak)** runs the role-leak checks for the citizen and public routes and reviews /me/* and /public/* for personal data.
- **Support (Shlok)** re-runs the tamper and replay tests and hands the results to Siddh.
- Needs from: all routers stable at Sprint 3 Day 1; Dhruv's matrix results; Shlok's payment tamper results.
- Done when: no cross-role access and no personal data in public responses.

**S3-12 Production-like deployment and fallback (owner; Shlok and Dhruv support)**

- Do: final Neon, Render and Vercel configuration with all secret values; migrations on the production-like database; rollback tags; compose fallback; backup and seed scripts; cold-start mitigation. Have someone who did not write them deploy from the docs.
- **Owner (Siddh)** does the configuration, enters the final environment values, runs the migrations, sets the rollback tags and prepares the compose fallback and cold-start mitigation.
- **Support (Shlok)** writes the backup and restore script and the seed scripts.
- **Support (Dhruv)** deploys from the docs on a clean machine with dummy values (he did not write them) and times it.
- Needs from: Dhruv's clean-deploy result; Shlok's scripts.
- Done when: a fresh deploy takes 30 minutes or less.

**S3-15 and S3-18 (owner), S3-16 and S3-17 (support)**

- S3-15: assign who plays which role and run two rehearsals; Dhruv drafts the demo script from the scenario test. S3-18: freeze, tag, build from the tag, deploy, run smoke.
- S3-16 (owner Zalak): write the architecture and deployment overview and review the final report.
- S3-17 (owner Dhruv): approve the P0 list and decide what blocks the release.

## Zalak: Citizen and public backend

Builds everything a citizen touches on the server: account, report, tracking, information responses, abuse limits, and the duplicate-grouping endpoint. Also owns the LLM adapter (a GenAI task), the AI-versus-human analytics and the final report.

### Sprint 1

**S1-03 Citizen register and login API (owner; Dhruv supports tests)**

- Already in repo: POST /api/v1/auth/login already works for non-staff roles (no MFA, token from _complete_login); argon2 hashing and lockout in security.py; the NORMAL_USER role; no register endpoint.

- Do: add POST /api/v1/auth/register: validate name, email, phone and password policy; create users (NORMAL_USER, status ACTIVE) and user_auth rows; email verification is a documented stub (the link is logged); a duplicate email returns 409 with a generic message; add rate limiting; write an audit entry. Add a require_citizen dependency in auth.py. Tests (at least 8): success, duplicate, weak password, rate limit, login after register, and a citizen refused on /dm.
- **Owner (Zalak)** builds POST /api/v1/auth/register with validation, rate limiting and an audit entry, adds the require_citizen dependency and writes at least 8 tests.
- **Support (Dhruv)** provides the DB test fixtures (drch_test database, citizen and staff users) so Zalak's tests actually execute; Siddh's CI job runs them and fails if no DB test ran.
- Needs from: Siddh contract and migration 04 merged; Dhruv's DB test fixtures and Siddh's CI job that actually runs DB tests.
- Gives to: Janvi (auth UI), and your own S1-04.
- Done when: a citizen can register and log in against the real database and cannot reach /dm.

**S1-04 Citizen report API and my incidents (owner; Nisarg supports)**

- Already in repo: the incidents table (idempotency_key unique per reporter, geom, a CHECK that location or address exists), incident_status_history, the DM state machine, the Scenario literal in common.py. Also in repo: a SQLite stand-in merged on 17 Oct (backend/main.py, database.py, models.py, test_citizen_api.py) with POST and GET /api/citizen/reports that trust an X-User-Id header from the client. It has no verified identity and does not use the real database.
- Do: POST /api/v1/incidents requires a citizen and an Idempotency-Key header; body has scenario, description, people_affected, reported_urgency, location_source, latitude and longitude or an address, landmark, details. Check the point with Nisarg's geo.py (422 OUTSIDE_GUJARAT), set district, create incident_ref, write the first history row. GET /me/incidents lists own incidents. GET /me/incidents/{id} returns the timeline from incident_status_history without internal rationale or dispatcher notes; someone else's id returns 404.
- **Owner (Zalak)** builds POST /api/v1/incidents with the Idempotency-Key, the Gujarat check, district, incident_ref and first history row, plus GET /me/incidents and the timeline (someone else's id gives 404). Retires the SQLite stand-in once Janvi's UI calls the new API: removes /api/citizen/*, database.py, models.py and the root-level test file (or switches them off by default), so no header-trusted route exists on staging.
- **Support (Nisarg)** delivers geo.py by Day 2 and reviews the idempotency and ownership SQL.
- Needs from: Nisarg's geo.py (a bounding-box stub is fine by Day 2); Siddh migration 04; Janvi agrees the request and response shape and moves the UI off /api/citizen/* before the stand-in is removed.
- Gives to: Janvi (report UI), Tanishq and Siddh (dispatcher console testing).
- Done when: the same key returns the same incident, a point outside Gujarat is rejected, and a citizen sees only their own. No route accepts an identity from an X-User-Id header.

**S1-02 Migrations (support, owner Siddh)**

- Do: list the new incident columns and enum values the report API needs and review migration 04.

**S1-09 Dispatcher console on real data (support, owner Tanishq)**

- Do: fix the backend defects that come from citizen-created incidents in the incident and history code (for example assumptions about the reporter), after Tanishq's defect list. Shlok fixes the ones in the /dm routes. Keep tests green.
- Needs from: Tanishq's defect list.

## Sprint 2

**S2-08 Optional LLM adapter (GenAI) (owner; Siddh supports for key storage)**

- Already in repo: ai-workers/requirements.txt lists langchain, langgraph, groq, google-genai and ollama, but nothing uses them. Rudra's /triage service exists from Sprint 1.
- Do: write ai-workers/service/llm.py with a provider switch behind LLM_ENABLED and a key in env. Ask for JSON limited to the allowed labels and validate it. Use it only for a second opinion on low-confidence cases and for an English summary of Hindi or Gujarati text. Never send the reporter's identity. Write tests with a stub LLM.
- **Owner (Zalak)** writes ai-workers/service/llm.py with the provider switch behind LLM_ENABLED, validated JSON limited to the allowed labels, and the stub-LLM tests. The key is read from an environment variable only.
- **Support (Siddh)** decides where provider keys live (hosting environment variables, never git), enters the key in staging and keeps LLM_ENABLED off by default.
- **AI-part sign-off (Neeti)**: reviews the contract and the outputs before merge.
- Needs from: Neeti for the allowed labels and the /triage contract; Siddh for how keys are stored and for entering them in the hosting consoles. Free tiers may let the provider use inputs, so only public or synthetic text is used in the demo.
- Gives to: Neeti (S3-07 evaluation).
- Done when: it is off by default, at least 99% of responses are schema-valid, and no personal data is in any prompt.

**S2-13 Citizen information response, polish, abuse limits (owner; Janvi supports)**

- Already in repo: DM POST …/information-requests puts an incident into NEEDS_INFORMATION; there is no way for the citizen to answer; no rate limiting beyond login lockout.
- Do: POST /api/v1/me/incidents/{id}/information-response, allowed only for your own incident in NEEDS_INFORMATION; store the text or corrected location, return the incident to SUBMITTED and write history. Replace raw status codes in the timeline with readable labels. Add rate limits on register, login failures and report submission; return 429 without revealing the thresholds.
- **Owner (Zalak)** builds the information-response endpoint, readable timeline labels and the rate limits on register, login failures and report submission (429 without revealing thresholds); Siddh reviews the limiter approach.
- **Support (Janvi)** builds the response form in My reports for an incident in Needs information.
- Needs from: Janvi builds the response form; Siddh reviews the limiter approach.
- Done when: answering outside NEEDS_INFORMATION is refused and limits return 429.

**S2-07 Duplicate grouping v2 (support, owner Dhruvil)**

- Already in repo: GET /dm/incidents/{id}/duplicates in routers/dm/incidents.py (same scenario, radius, time window, same address; returns method rules), the human LINK, MERGE or DISMISS endpoint, duplicates-panel.tsx.
- Do: call Dhruvil's text-similarity module on the candidates the existing query already finds, combine its score with distance, time and scenario using the weights Dhruvil tunes, and return method rules+text with a plain-language explanation. Keep human decisions exactly as they are. Never auto-merge. Add tests.
- Needs from: Dhruvil's module and labelled pairs by Day 2; Tanishq to show the score in duplicates-panel.tsx.

**S2-14 Test hardening (support, owner Dhruv)**

- Do: write DB-backed tests for citizen, public and duplicate routes, and contract tests against the OpenAPI document.

## Sprint 3

**S3-06 AI-versus-human analytics (owner; Tanishq and Neeti support)**

- Already in repo: ai_recommendations stores human_action, human_rationale and decided_at when a dispatcher decides; /dm/analytics and AnalyticsView exist.
- Do: add an endpoint (or extend analytics) counting accepted, modified and rejected AI suggestions by scenario and confidence band; document the definitions.
- **Owner (Zalak)** builds the endpoint counting accepted, modified and rejected AI suggestions by scenario and confidence band and documents the definitions.
- **Support (Neeti)** names the confidence bands.
- **Support (Tanishq)** draws the chart from Zalak's endpoint.
- Needs from: Neeti names the confidence bands; Tanishq draws the chart; rehearsal data from S3-15 so the numbers are real.

**S3-16 Final report and documentation (owner; every member and Siddh support)**

- Do: assemble the final report: architecture, API docs, ER update, test notes and each member's section (citizen and public API, dispatch and field API, payments with the sandbox limits, ingestion and sources, AI evaluation report, dataset and labelling, frontend, dispatcher and public UI guide, QA).
- **Owner (Zalak)** assembles the report from the sections each member writes, keeps the API docs and the ER diagram in line with the final schema, and checks that every section says what is real and what is simulated.
- **Support (Siddh)** writes the architecture and deployment overview and reviews the final report.
- **Support (every member)** writes their own section, as listed in their S3-15, S3-16, S3-17 entry.
- Needs from: each member's section by Sprint 3 Day 4; Siddh's overview.
- Done when: the report is complete, matches the tagged release, and every section is signed off by its author.

**S3-07 LLM evaluation (support, owner Neeti)**

- Do: run the LLM adapter over the low-confidence subset that Neeti and Dhruvil define, using only public or synthetic text, and hand the outputs to Neeti.
- Needs from: Neeti's subset and Dhruvil's model-side predictions.

**S3-01 and S3-02 (support)**

- S3-01: write the API-level steps for the citizen and public parts of the scenario test. S3-02: run the role-leak checks for citizen and public routes and review /me/* and /public/* for personal data.

**S3-15 and S3-17 (team-wide)**

- Rehearsal: play the citizen role. Defect buffer: own bugs in the citizen, public and duplicate routers; Dhruv triages.

## Nisarg: Dispatch and field-team backend

Owns the public API and geofence, the team-recommendation logic and team-request decisions, and the whole field-team server side. Also writes the Gujarat agencies and teams seed.

### Sprint 1

**S1-07 Public verified-incidents API and Gujarat geofence (owner; Siddh supports)**

- Already in repo: the incidents table with status, verified_by, geom and canonical_incident_id; the authenticated /dm/map endpoints as a SQL reference. Also in repo: backend/main.py serves /api/public/incidents, /api/public/alerts and /api/public/shelters from a SQLite demo seed (database.py).
- Do: create backend/app/geo.py: a simplified Gujarat polygon loaded at start-up, a fast bounding-box check, is_in_gujarat(lat, lon) and district_for(lat, lon). Create routers/public.py with GET /api/v1/public/incidents returning only incidents whose status is VERIFIED, DISPATCHED or RESOLVED, with verified_by set and not a linked duplicate. Return reference, scenario, district, status, verified time, priority and coordinates rounded to the precision fixed in the contract. No reporter fields and no free text. Filters: district, scenario, bounding box. Test that submitted, rejected and ingested records never appear.
- **Owner (Nisarg)** builds geo.py (polygon, bounding-box check, is_in_gujarat, district_for) and routers/public.py with GET /api/v1/public/incidents, its filters and the security test. Hands geo.py to Zalak and the response format to Tanishq by Day 2. Removes or switches off the SQLite public incidents and alerts routes once the new router is live (Siddh decides what happens to shelters, which is out of scope).
- **Support (Siddh)** reviews the SQL (status filter, verified_by requirement, coordinate rounding) and provides the Gujarat boundary source and the district list.
- Needs from: Siddh migration 04 and the district list; Zalak needs geo.py by Day 2; Tanishq needs the response format by Day 2.
- Gives to: Zalak, Tanishq.
- Done when: the security test passes and a verified seed incident appears. The SQLite public routes are gone or off by default.

**S1-11 Team-recommendation rules and prototype query (owner)**

- Already in repo: resources (type TEAM, capability JSON, status, current_geom, version), GET and POST /dm/resources, row-locked reservations.
- Do: define a rules table from scenario to required capabilities (for example flood to boat and rescue, fire to fire tender, earthquake to search-and-rescue and medical), using the same capability keys as the seed. Write the PostGIS query that ranks available teams by capability match, availability and ST_Distance, and returns the reasons for each rank. Test on the seed incidents.
- Needs from: Siddh seeds the teams with capability keys that you agree on Day 1 or 2.
- Gives to: S2-02.
- Done when: for 5 seed incidents the top 3 teams are sensible and unavailable teams are excluded.

**S1-02 Migrations and Gujarat seed (support, owner Siddh)**

- Do: write the Gujarat agencies (verify the names) and at least 10 teams as TEAM resource rows with coordinates and capability keys, agree the keys with Siddh by Day 1, and hand the seed SQL to Siddh by Day 2.

**S1-04 Citizen report API (support, owner Zalak)**

- Do: deliver geo.py to Zalak by Day 2 and review the idempotency and ownership SQL.

**S1-09 Dispatcher console (support, owner Tanishq)**

- Do: run the dispatch flow on citizen incidents (create, acknowledge as staff, escalation) and report defects.

## Sprint 2

**S2-01 Field-team backend (owner; Shlok and Dhruv support)**

- Already in repo: dispatch routes in routers/dm/dispatches.py (acknowledgement, rejection, progress, completion, failure); STAFF_ROLES contains only DISASTER_MGMT and APP_MGMT, so a FIELD_TEAM user logs in with a password and no MFA; Principal and require_dm in auth.py; cli.py create-staff.
- Do: add a require_field dependency that loads the user's team through field_team_members. Add a CLI command create-field for a team. Create routers/field.py: GET /field/me; GET /field/assignments (dispatches whose assignments include the user's team, with incident summary and instructions); POST /field/dispatches/{id}/acknowledgement, rejection (needs a reason), progress (EN_ROUTE or ON_SCENE plus a note, stored in dispatch_progress), team-requests (capability and note, stored OPEN) and completion (summary; dispatch becomes COMPLETED, the incident stays open and is flagged for resolution). Reuse the transition functions Shlok refactors. Write audit entries. Another team's dispatch returns 404.
- **Owner (Nisarg)** adds require_field and the create-field CLI command, builds routers/field.py (me, assignments, acknowledgement, rejection, progress, team-requests, completion), writes audit entries, returns 404 for another team's dispatch and writes at least 12 tests including cross-team isolation.

- **Support (Shlok)** refactors services/dispatch.py so the dispatcher routes and the field routes call the same state-transition functions, keeping the audit chain intact, and merges it by Sprint 2 Day 1, before Nisarg builds on it. Shlok wrote the DM module, so he knows these routes best; Siddh reviews the change.
- **Support (Dhruv)** sets up the test fixtures for field users.
- Needs from: Siddh migration 04 and Shlok's refactor of shared transitions; Dhruv's DB test setup; Janvi and Tanishq agree the contract on Day 1.
- Gives to: Janvi (field UI), Tanishq (dispatcher UI), Dhruv (tests).
- Done when: at least 12 tests pass, including cross-team isolation, and completion does not close the incident.

**S2-02 Team recommendation endpoint (owner; Siddh supports)**

- Do: GET /dm/incidents/{id}/team-recommendations using the S1-11 query; return ranked teams with reasons and a label saying it needs dispatcher confirmation. It never creates a dispatch. Dispatch creation stays on the existing route with the DISPATCH permission. Nisarg also builds the team-request decisions.
- **Owner (Nisarg)** builds GET /dm/incidents/{id}/team-recommendations on the S1-11 query, returning ranked teams with reasons and the needs-confirmation label; it never creates a dispatch. Also builds the team-request decision endpoints: approve creates a pre-filled dispatch draft, decline stores a rationale, both need the DISPATCH permission.
- **Support (Siddh)** reviews the permission checks, audit rows and idempotency of the decision endpoints.
- Needs from: S1-11 and the seed; Tanishq builds the panel.
- Done when: a recommendation never creates a dispatch and double-booking is still refused. Approving only creates a draft, and declining stores a rationale.

**S2-03 Dispatcher UI (support, owner Tanishq)**

- Do: answer API questions and fix contract problems the same day; provide Swagger examples.

**S2-14 Test hardening (support, owner Dhruv)**

- Do: write cross-team isolation tests for the field router.

## Sprint 3

**S3-13 Performance smoke (owner; Shlok supports)**

- Do: write a light k6 or Locust script for the public map and report submit; run it against staging; read EXPLAIN for the public incidents and recommendation queries; add an index by migration only if the plan shows a need.
- **Owner (Nisarg)** reads EXPLAIN for the public incidents and recommendation queries and proposes an index only if the plan shows a need.
- **Support (Shlok)** writes the k6 or Locust script for the public map and report submit, runs it against Siddh's staging and records the p95 for the public map endpoint.
- Needs from: Siddh's staging; Siddh to merge any migration.
- Done when: the p95 for the public map endpoint is recorded.

**S3-14 Team-recommendation tuning (owner)**

- Do: with two reviewers, judge the top 3 teams for 10 incidents and adjust the rules.
- Needs from: Siddh's seed; the demo scenario list from S3-15.

**S3-04 About, usage and safety content (support, owner Tanishq)**

- Do: check the cyclone, flood, earthquake and fire guidance against official pages and add the source for each.
- Needs from: Tanishq's draft of the content pages.

**S3-01 Scenario test (support), S3-15, S3-16, S3-17 (team-wide)**

- Do: write the API steps for assignment and the field flow; rehearse; write the dispatch and field API section; own bugs in field, public and recommendation code.

## Shlok: Payments, signal promotion and dispatch refactor (full-stack)

Owns the donation flow from the campaign list to a verified receipt, on the backend and on the donation screens. Also owns the signal promotion API and the shared dispatch-transition refactor, because he wrote the DM module.

### Sprint 1

**S1-16 Payments foundation (owner; Siddh supports for gateway keys)**

- Already in repo: donations (status PENDING, CONFIRMED or FAILED; unique idempotency_key; unique gateway name plus transaction reference; receipt_no allowed only when CONFIRMED), donation_campaigns, and the DM oversight and campaign pages. There is no gateway code.
- Do: create backend/app/payments/ with a PaymentGateway interface (create_order, verify_payment, parse_webhook). Write MockGateway: it signs payments with a local secret using HMAC and every response says SANDBOX. Find out whether a free test-mode account with a real gateway (Razorpay or Stripe) can be opened this week and write down the answer; if yes, add an adapter stub behind the same interface. With Siddh, agree the migration 04 columns (gateway_order_id, gateway_payment_id, verified_at). Build GET /api/v1/public/campaigns (ACTIVE only) and POST /api/v1/donations (needs an Idempotency-Key; amount above zero, currency INR, campaign ACTIVE; a logged-in citizen first, a guest with email only if time allows since the schema supports it). Write tests.
- **Owner (Shlok)** builds the payments package, the PaymentGateway interface, MockGateway, the campaigns API, create-order and the tests; writes up whether a free test-mode account is possible and adds the adapter stub. All keys are read from environment variables; dummy values are used locally.
- **Support (Siddh)** (secrets only) if a test-mode gateway account can be opened, opens it, keeps the keys and enters them in the hosting consoles; also sets the MockGateway signing secret on staging. Agrees the migration 04 columns with Shlok.
- Needs from: Siddh migration 04 and the contract; Siddh's CI job and Dhruv's fixtures that run DB tests.
- Gives to: Janvi (donation screens in Sprint 2) and your own S2-12.

- Done when: create-order returns an order id, the same key returns the same donation, only ACTIVE campaigns are listed, and no secret is in the repo.

**S1-02 Migrations and Gujarat seed (support, owner Siddh)**

- Do: write the seed rows for the donation campaigns and about 20 demo incidents across Gujarat districts (mix of statuses, scenarios and priorities, some simulated) once the migration 04 columns are agreed, and hand the seed SQL to Siddh by Day 3.
- Needs from: Siddh migration 04; Zalak's column list.

**S1-09 Dispatcher console on real data (support, owner Tanishq)**

- Do: fix the defects in the /dm routes that Tanishq finds when the console runs on citizen-created incidents. Keep tests green.
- Needs from: Tanishq's defect list.

## Sprint 2

**S2-01 Shared dispatch-transition refactor (support, owner Nisarg)**

- Already in repo: services/dispatch.py and routers/dm/dispatches.py, which Shlok wrote as part of the DM module.
- Do: refactor services/dispatch.py so the dispatcher routes and the new field routes call the same state-transition functions, keeping the audit chain intact. Merge it by Sprint 2 Day 1, before Nisarg builds on it. Existing dispatch tests must still pass.
- Needs from: Siddh's review as a high-risk change.
- Gives to: Nisarg.
- Done when: dispatcher routes and field routes share the same transition functions and the existing tests pass.

**S2-10 Signal promotion API (owner; Rudra and Tanishq support, Siddh reviews)**

- Already in repo: incidents require a reporter, so ingested items need the system user from S1-02; audit.record() writes the audit trail.
- Do: GET /dm/signals (list, filter by status). POST /dm/signals/{id}/promote creates a SUBMITTED incident (source and is_simulated copied, location from the signal) in one transaction with an audit row. POST dismiss needs a rationale. Confirm the public API still returns nothing until a dispatcher verifies the promoted incident.
- **Owner (Shlok)** builds GET /dm/signals, the promote endpoint (one transaction, audit row) and dismiss with rationale, plus the negative test that a promoted signal is not public before verification.
- **Support (Rudra)** fills source_signals with text, language, location and AI label, makes sure the fields match what promote needs, and supplies sample rows by Sprint 2 Day 2.
- **Support (Tanishq)** builds the /dm/signals queue page with promote, dismiss and the AI pre-label.
- Needs from: Rudra fills source_signals with text, language, location and AI label; Tanishq builds the queue page; Neeti explains what the label field means; Siddh reviews the transaction and audit row.

- Gives to: Tanishq, and the S3-01 scenario test.
- Done when: a negative test proves a promoted signal is not public before verification.

**S2-12 Donations end to end (owner; Janvi, Siddh and Dhruv support)**

- Already in repo: your S1-16 interface and MockGateway; the BFF session pattern in the frontend; dm/campaigns for staff.
- Do: POST /donations/{id}/verify takes the payment id and signature from the client. The server recomputes the signature with the gateway secret, checks order, amount and currency, then sets CONFIRMED, receipt_no and receipt_issued_at in one transaction. The browser saying success is never trusted. Webhook POST /payments/webhook/{gateway}: verify the signature, ignore repeated event ids, and cope with the webhook arriving before the client callback. GET /me/donations with the receipt. Screens: campaign list, donate form, checkout (mock window or the gateway test checkout), and a result page showing PENDING, CONFIRMED or FAILED with a SANDBOX banner. Tests: tampered signature, replay, wrong amount, webhook first.
- **Owner (Shlok)** builds POST /donations/{id}/verify (server-side signature check, one transaction), the webhook endpoint, GET /me/donations, the checkout screen and the tests (tampered signature, replay, wrong amount, webhook first).
- **Support (Janvi)** builds the campaign list, donate form and result page (PENDING, CONFIRMED, FAILED, SANDBOX banner) against Shlok's endpoints and shares the session and form components.
- **Support (Dhruv)** keeps the payment tests in the test run and checks that the staging webhook URL answers.
- **Support (Siddh)** reviews the signature-verification code and enters the gateway and webhook secrets in staging.
- Needs from: Janvi shares form components and the session pattern and builds part of the screens; Siddh provides the webhook URL and puts the secrets into the staging env; Siddh reviews the verification code.
- Done when: CONFIRMED happens only after server-side verification, a tampered signature is rejected, and a repeated webhook changes nothing.

## Sprint 3

**S3-11 Payments hardening (owner; Dhruv and Janvi support)**

- Do: cover every ending: declined, abandoned, timeout, webhook before callback, duplicate webhook. Add the receipts view and SANDBOX labels everywhere. Check the DM donation oversight page on real data.
- **Owner (Shlok)** covers every ending (declined, abandoned, timeout, webhook before callback, duplicate webhook), the receipts view, SANDBOX labels everywhere and the DM donation oversight check.
- **Support (Dhruv)** re-runs the payment matrix on staging; Siddh keeps it in the CI run.
- **Support (Janvi)** builds the UI states for declined, abandoned and timed-out payments.
- Needs from: Dhruv re-runs the matrix on staging; Janvi does the UI states.
- Done when: every path ends in PENDING, CONFIRMED or FAILED correctly and nothing is CONFIRMED without verification.

**S3-02 Security review (support, owner Siddh)**

- Do: re-run the tamper and replay tests and hand the results to Siddh.

**S3-12 and S3-13 (support)**

- S3-12 (owner Siddh): write the backup and restore script and the seed scripts.
- S3-13 (owner Nisarg): write the k6 or Locust script for the public map and report submit, run it against staging and record the p95 for the public map endpoint.

**S3-15, S3-16, S3-17 (team-wide)**

- Rehearsal: run the donation part. Docs: write the payments section, including the sandbox limits. Defect buffer: own payment bugs.

## Rudra: Ingestion, external alerts and AI service skeleton

Owns every automatic data source: official alerts, the social source, the scheduler, and the health of all of it. Also builds the AI service skeleton and the adversarial post sets (ML work).

### Sprint 1

**S1-15 Ingestion framework, USGS adapter, fixtures, source spike (owner; Siddh supports)**

- Already in repo: the 60-second background loop in main.py (the pattern to copy); httpx in requirements; the ingestion tables arriving in migration 04. The README mentions a SACHET parser that was never written.
- Do: create backend/app/ingestion/ with an Adapter base (fetch, normalise, upsert), run logging into ingest_runs, and a unique key on (source, source_uid) so repeats never duplicate. Write the USGS earthquake adapter using USGS's public feed filtered to a Gujarat bounding box, writing to external_alerts (event type, severity from magnitude, area text). Write a fixtures loader that marks every row is_fallback. Add POST /api/v1/internal/ingest/run guarded by a token in the INGEST_TOKEN env var. Spike and write a short memo for each source: SACHET CAP RSS (items link to CAP XML), GDACS or IMD (optional), and one social source (for example Mastodon, Bluesky, Reddit or X). For each: can we reach it from a laptop and from Render, is a key needed, what do its terms allow.
- **Owner (Rudra)** builds the adapter base, ingest_runs logging, the unique key on (source, source_uid), the USGS adapter, the fixtures loader flagged is_fallback, POST /api/v1/internal/ingest/run guarded by INGEST_TOKEN (read from the environment; the real value is set by Siddh) and the source memos.
- **Support (Siddh)** runs each source once from the Render environment and reports the reachability result to Rudra by Day 3.
- Needs from: Siddh table definitions in migration 04; Siddh runs the reachability check from the Render environment and gives you the result.
- Gives to: your own S2-09, Siddh (signals), Janvi (alerts layer).
- Done when: running twice creates no duplicates, failures are recorded and not raised, fixtures are flagged, and the memo exists.

**S1-13 AI service skeleton (owner; Neeti, Dhruvil and Siddh support)**

- Already in repo: services/ai_provider.py defines what a triage service must return (model, scenario, priority, confidence, uncertainty, factors) and calls it with a 10-second timeout; ai-workers/model/app.py is an image-only FastAPI service; ai-workers/Dockerfile and the compose service exist; ai-workers/worker.py is a single line.
- Do: create ai-workers/service/main.py with POST /triage that accepts the payload built in routers/dm/ai.py (incident_ref, reported_scenario, description, reported_urgency, people_affected, details, has_coordinates) and returns the contract. Use the rules baseline inside. Add GET /health, a Dockerfile and a compose entry, and set AI_TRIAGE_URL in .env.example.
- **Owner (Rudra)** creates ai-workers/service/main.py with POST /triage and GET /health on the rules baseline. Until Neeti's rules.py lands he starts from the keyword rules in the frontend aiTriage.ts.
- **Support (Neeti)** ports the keyword rules to ai-workers/service/rules.py in S1-12, hands it over by Day 2 and confirms the contract fields with Siddh.
- **Support (Dhruvil)** writes the Dockerfile and the tests for /triage.
- **Support (Siddh)** confirms the contract, adds the compose entry and the CI and CD image build, and sets AI_TRIAGE_URL in .env.example.
- Needs from: Neeti's rules.py by Day 2; Siddh's confirmed contract; Dhruvil's Dockerfile.
- Gives to: Neeti (S2-06 builds on it), your own S2-09 (pre-labels), Zalak (S2-08).
- Done when: the DM Request AI triage button returns a suggestion on seed incidents, and stopping the service gives the unavailable path.

**S1-10 Text dataset (support, owner Neeti)**

- Do: collect citizen-style and social-style posts (public posts from the source you spike in S1-15 with the source recorded, plus posts written in English, Hindi and Gujarati) and fill the spreadsheet. Label about 200 texts with Neeti's guideline, including the 60 shared texts that all three ML members label. Mark anything unclear.
- Needs from: Neeti's guideline and spreadsheet on Day 1.

## Sprint 2

**S2-09 Ingestion: SACHET, social or replay, scheduler, health (owner; Neeti and Siddh support)**

- Do: CAP parser: read the RSS list, fetch each CAP XML item, keep items whose area text or polygon falls in Gujarat (use Nisarg's geo.py), and store severity, urgency, certainty, effective and expiry times and language. Social adapter: use the authorised source chosen in the spike, or a replay dataset marked simulated; write to source_signals with text, language, location if any, and an AI label obtained by calling Neeti's /triage (rules until S2-06 is done). Scheduler: add a loop at start-up in main.py that runs all adapters every 10 minutes, catches every exception and logs each run. Keep the token endpoint for an outside cron and add a Poll-now action for dispatchers and admins.
- **Owner (Rudra)** builds the CAP parser, the social or replay adapter writing source_signals, the 10-minute scheduler in main.py, the token endpoint and the Poll-now action.
- **Support (Neeti)** exposes /triage so Rudra can pre-label signals and explains what the label fields mean.

- **Support (Siddh)** owns the cron trigger workflow (S2-16) and adds INGEST_TOKEN and the other environment values to the host and to the GitHub Actions secrets.
- Needs from: Neeti gives the /triage contract for pre-labels; Siddh sets up the cron trigger and adds the secrets to the host and GitHub Actions; Siddh confirms the signals table; Janvi consumes the alerts API.
- Done when: the scheduler runs unattended for at least 6 cycles locally and every record carries its source and fallback flag.

**S2-10 Signal promotion (support, owner Shlok)**

- Do: make sure the source_signals fields match what the promote endpoint needs; supply sample rows.

**S2-11 Public alerts API (support, owner Janvi)**

- Do: build GET /api/v1/public/alerts: official sources only, not expired, with source name and is_fallback. Janvi draws the layer.

**S2-06 AI integration (support, owner Neeti)**

- Do: write 10 prompt-injection posts (for example posts that tell the model to verify an incident or to ignore its rules), run them against /triage and report any that change a label outside the allowed set. They are the starting point of the S3-10 adversarial set.

## Sprint 3

**S3-09 Ingestion soak test and health UI (owner; Siddh supports)**

- Do: run the scheduler for 24 hours on staging or the demo host. Build a small health panel: last run per source, counts and last error.
- **Owner (Rudra)** runs the scheduler for 24 hours and builds the health panel (last run per source, counts, last error).
- **Support (Siddh)** provides the host and the monitoring for the soak.
- Needs from: Siddh provides the host and monitoring; Tanishq can help with the panel layout.
- Done when: at least 95% of scheduled runs complete and no duplicate alerts exist.

**S3-10 Social source final and signal sanitisation (owner)**

- Do: finish the authorised source or the labelled replay. Clean every text before use: strip HTML and links, cap the length, detect the language. Stay prompt-injection safe: never put raw post text inside instructions, use delimiters, and make the classifier output one label from a fixed set. Build an adversarial set of posts and test it. Start from the ten prompt-injection posts of S2-06.
- **AI-part sign-off (Neeti)**: reviews the label set, the sanitisation and the adversarial results before merge.
- Needs from: Neeti's /triage and label set; Dhruvil's help with adversarial examples if available.
- Done when: adversarial posts cannot change a label outside the allowed set or trigger any action.

**S3-15, S3-16 (team-wide)**

- Rehearsal: run the ingestion steps. Docs: write the ingestion and sources section, saying which sources were real and which were fallback.

## Neeti: AI lead

ML, and available for GenAI work. Owns dataset design, the text model, the evaluation, the /triage integration and the LLM-versus-model decision. Leads the AI part and signs off AI changes made by Dhruvil, Rudra and Zalak (the LLM adapter) before merge.

### Sprint 1

**S1-10 Text dataset (owner; Dhruvil and Rudra collect and label)**

- Already in repo: nothing for text. The image dataset in ai-workers/model is a separate 3.2 GB download.
- Do: write the label schema (cyclone, flood, fire, earthquake, other_disaster, not_disaster) and a one-page guideline with Gujarat examples and edge cases (for example a heatwave or landslide is other_disaster; an opinion post is not_disaster). Create the spreadsheet template (text, label, lang, source, synthetic, event_id). Decide where texts come from: public headlines and official advisories with citations, citizen-style posts written by the team, and synthetic ones that are flagged. Hand out labelling batches. Keep the test set human-written only and split by event and template so test never shares them with train. Write the validation script (schema, duplicates, leakage). Optional, only if labelling falls short: Zalak, Tanishq or Siddh may generate synthetic texts with an LLM (GenAI work), flagged synthetic, training split only, never in the test set; Neeti reviews their labels.
- **Owner (Neeti)** writes the label schema and one-page guideline with Gujarat examples, the spreadsheet template, decides the text sources, hands out batches, keeps the test set human-written and split by event and template, and writes the validation script. Also labels about 200 texts, including the 60 shared texts that all three ML members label.
- **Support (Dhruvil)** collects and cleans the public headlines (with source) and the official advisories, fills the spreadsheet including language and source, and labels about 200 texts (60 shared).
- **Support (Rudra)** collects citizen-style and social posts from the source in his S1-15 spike, fills the spreadsheet, and labels about 200 texts (60 shared), marking anything unclear.
- Needs from: Dhruvil collects and cleans headlines and advisories; Rudra collects citizen-style posts; Neeti, Dhruvil and Rudra label; Siddh reviews the licence of any public corpus.
- Gives to: S1-12, S2-05 and the pairs for S2-07.
- Done when: at least 600 labelled items, at least 20 per class in test, agreement on the 60 items labelled by all three reported, and no leakage.

**S1-12 Baselines and evaluation harness (owner; Dhruvil supports)**

- Already in repo: the frontend aiTriage.ts (keyword and priority rules) and duplicateDetection.ts; there is no Python baseline.
- Do: port the keyword rules to Python (ai-workers/service/rules.py). Train TF-IDF with logistic regression as v0. Write the evaluation script that prints macro-F1, per-class precision and recall, the confusion matrix, per-language results and latency. Commit the results under docs/ai.

- **Owner (Neeti)** ports the keyword rules to ai-workers/service/rules.py, trains the TF-IDF plus logistic regression v0, writes the evaluation script and commits the results under docs/ai. Hands rules.py to Rudra by Day 2 for the service skeleton.
- **Support (Dhruvil)** writes the helper code for loading splits, metric tables and confusion-matrix plots.
- Needs from: partial data (about 200 items) by Day 2.
- Done when: one command prints the baseline table on the validation split.

**S1-13 AI service skeleton (support, owner Rudra)**

- Do: hand over ai-workers/service/rules.py (from S1-12) by Day 2 and confirm the contract fields with Siddh and Rudra.
- Needs from: Siddh's confirmed contract.

## Sprint 2

**S2-05 Text model v1 (owner; Dhruvil supports)**

- Do: train a TF-IDF character n-gram classifier and a multilingual-embedding plus logistic regression candidate; compare macro-F1, latency and memory; pick one; set the confidence threshold; write the model card; save the artifact. Look at the test set once, at the end.
- **Owner (Neeti)** trains the TF-IDF character n-gram classifier and the multilingual-embedding candidate, compares them, picks one, sets the confidence threshold, writes the model card and saves the artifact; looks at the test set once, at the end.
- **Support (Dhruvil)** runs the embedding experiments and produces the per-class error tables.
- Needs from: the frozen data from S1-10 and the harness from S1-12; Dhruvil runs embedding experiments and error tables.
- Done when: it meets the targets (macro-F1 at least 0.75 and at least 0.10 above rules; per-class recall at least 0.60) or the miss is written down.

**S2-06 AI integration (owner; Siddh, Tanishq, Dhruvil and Rudra support)**

- Already in repo: the config setting for the low-confidence threshold; the guardrails and human-decision flow; ai-panel.tsx.
- Do: make /triage use the model. Write the priority rules down (people affected, trapped or medical words, source reliability, class). Return LOW_CONFIDENCE below the threshold. Agree with Dhruvil how the in-process fallback loads the same model. Run the guardrail tests and the 10 prompt-injection posts. Agree with Tanishq what the panel shows.
- **Owner (Neeti)** makes /triage use the model, writes down the priority rules, returns LOW_CONFIDENCE below the threshold, agrees with Dhruvil how the in-process fallback loads the same model, runs the guardrail tests and the 10 prompt-injection posts, and agrees the panel contents with Tanishq.
- **Support (Siddh)** makes sure the AI service image builds and publishes in CI and CD and decides where the model artifact is stored.
- **Support (Tanishq)** changes ai-panel.tsx to show "AI suggestion, not verified", confidence, factors, the out-of-scope message and the unavailable state.

- **Support (Dhruvil)** adds InProcessProvider in ai_provider.get_provider() (used when AI_TRIAGE_URL is empty and a model file exists), makes sure EARTHQUAKE is accepted end to end and extends backend/tests/test_ai.py.
- **Support (Rudra)** writes the 10 prompt-injection posts and runs them.
- Needs from: Siddh decides where the model artifact is stored; Tanishq updates the panel; Dhruvil builds InProcessProvider; Rudra writes the injection posts.
- Done when: the AI is evaluated and integrated by Sprint 2 Day 5, all outputs pass the guardrails, and the AI still cannot change an incident.

**S2-07 Duplicate grouping v2 (support, owner Dhruvil)**

- Do: review the text-similarity features and the threshold that Dhruvil picks from the labelled pairs, and sign them off as AI lead by Day 4.

**S2-08 Optional LLM adapter (support, owner Zalak)**

- Do: give Zalak the allowed labels and the /triage contract, and review the adapter's contract and outputs before merge (AI-part sign-off).

**S2-09 Ingestion (support, owner Rudra)**

- Do: expose /triage so Rudra can pre-label signals and tell Rudra what the label fields mean.

## Sprint 3

**S3-05 AI error analysis, thresholds, final model card (owner; Dhruvil supports)**

- Do: break errors down by class and language, re-tune the threshold, and write the final model card with its limits. Report numbers as measured.
- **Owner (Neeti)** breaks errors down by class and language, re-tunes the threshold and writes the final model card with its limits, with numbers measured on the untouched test set.
- **Support (Dhruvil)** produces the error tables.
- Needs from: Dhruvil helps with error tables; real dispatcher decisions from rehearsals.
- Done when: final numbers are reported on the untouched test set and the failure modes are listed.

**S3-07 LLM evaluation and decision (owner; Zalak and Dhruvil support)**

- Do: compare the LLM with the model on the low-confidence subset and record the decision.
- **Owner (Neeti)** defines the low-confidence subset with Dhruvil, computes macro-F1 for the model and for the LLM on it, and records the decision as AI lead.
- **Support (Zalak)** runs the LLM adapter over the subset, using only public or synthetic text, and hands the outputs to Neeti.
- **Support (Dhruvil)** produces the model predictions and the per-class tables for the subset.
- Needs from: Zalak's LLM outputs; Dhruvil's model-side tables.
- Done when: the LLM is enabled only if it improves macro-F1 on that subset by at least 5 points within the free quota; otherwise it stays off.

**S3-06 AI-versus-human analytics (support, owner Zalak)**

- Do: name the confidence bands for Zalak's analytics.

**S3-15, S3-16, S3-17 (team-wide)**

- Rehearsal: run the AI parts. Docs: write the AI evaluation report. Defect buffer: own AI service bugs.

## Dhruvil: ML (image model, similarity model, in-process loading)

Gets the data and model assets ready, builds the image path, the duplicate text-similarity model and the in-process model loading, and produces the labelled pairs for duplicates. Takes no LLM or non-ML work.

### Sprint 1

**S1-14 Image classifier reproduction (owner)**

- Already in repo: ai-workers/model/app.py serves POST /predict with seven classes (earthquake, fire, flood, hurricane, landslide, other_disaster, not_disaster), a top 3 and a needs_review flag; check_model.ipynb; the training script and notebook. The weights file best_model.pt is not in the repo; the README links to it on Google Drive.
- Do: download best_model.pt and classes.json from the README link (ask the person who trained it if the link fails), run the service locally and try at least 50 images. Compare with the README accuracy on a sample and write a short note. Write a mapping module: fire to INDUSTRIAL_FIRE, flood to URBAN_FLOODING, hurricane to CYCLONE, earthquake to EARTHQUAKE; landslide and other_disaster are out of scope; not_disaster gives no suggestion.
- **AI-part sign-off (Neeti)**: reviews the class mapping and the accuracy note before they are used.
- Needs from: Siddh for where to keep the model file (not in git); Neeti to know whether it will be used.
- Gives to: S2-08.
- Done when: at least 50 images are predicted and the accuracy note is written. If the weights cannot be obtained, tell Neeti and Siddh by Day 2 and mark image triage unavailable.

**S1-10 Text dataset (support, owner Neeti)**

- Do: collect and clean the public headlines (with their source) and the official advisories, fill the spreadsheet including language and source, and label about 200 texts with Neeti's guideline, including the 60 shared texts that all three ML members label. Mark anything unclear.
- Needs from: Neeti's guideline and template on Day 1.

**S1-12 Baselines (support, owner Neeti)**

- Do: write helper code for loading splits, metric tables and confusion-matrix plots.

**S1-13 AI service skeleton (support, owner Rudra)**

- Do: write the Dockerfile and the tests for /triage.

### Sprint 2

**S2-07 Duplicate grouping v2: text-similarity model and labelled pairs (owner; Zalak and Neeti support)**

- Already in repo: GET /dm/incidents/{id}/duplicates in routers/dm/incidents.py (same scenario, radius, time window, same address; returns method rules), the human LINK, MERGE or DISMISS endpoint, duplicates-panel.tsx, and the old client-side matcher in duplicateDetection.ts.
- Do: build at least 100 labelled pairs, at least 40 of them true duplicates, from the dataset, adding flagged near-duplicates where needed. Write the text-similarity function (character n-gram TF-IDF cosine, computed in Python on the candidate set only) as a module the backend can import. Tune the weights that combine it with distance, time and scenario, and choose the threshold, using the pairs. Report precision and F1 against the rules-only result.
- **Owner (Dhruvil)** delivers the labelled pairs CSV (id_a, id_b, label) and the similarity module with default weights by Sprint 2 Day 2, the tuned weights and threshold by Day 4, and the precision and F1 comparison.
- **Support (Zalak)** builds the endpoint change that uses the module, returns the score and the plain-language explanation, keeps human decisions as they are, never auto-merges, and adds the tests.
- **Support (Neeti)** reviews the features and the threshold and signs them off as AI lead by Day 4.
- Needs from: the dataset from S1-10; Zalak for the endpoint; Tanishq to show the score in duplicates-panel.tsx.
- Gives to: Zalak.
- Done when: precision is at least 0.80 and F1 is at least the rules-only result on the pair set.

**S2-05 Text model v1 (support, owner Neeti)**

- Do: run embedding experiments and produce per-class error tables.

**S2-06 AI integration (support, owner Neeti)**

- Do: add InProcessProvider in ai_provider.get_provider() (used when AI_TRIAGE_URL is empty and a model file exists), make sure EARTHQUAKE is accepted end to end, and extend backend/tests/test_ai.py. Agree with Neeti how it loads the same model.
- Needs from: Neeti's model artifact and final contract; Siddh for where the artifact is stored.

## Sprint 3

**S3-08 Image Analyse photo demo path (owner; Tanishq supports)**

- Do: let a dispatcher upload a photo on an incident and see an advisory label and confidence. Label it advisory; never store it as verification. First make /triage accept an optional image and call the image classifier through the S1-14 class mapping (this was part of S2-08 and moved here).
- **Owner (Dhruvil)** builds the photo-analyse path: an uploaded photo returns an advisory label and confidence, never stored as verification; dropped if the model is unavailable. Also adds the optional image input to /triage.
- **Support (Tanishq)** builds the upload control and result display on the incident page.
- Needs from: Tanishq builds the UI; the image input to /triage from this task. Drop it if the model is unavailable.

**S3-07 LLM evaluation and decision (support, owner Neeti)**

- Do: define the low-confidence subset with Neeti, and produce the model predictions and per-class tables for it.

**S3-05 AI error analysis (support, owner Neeti)**

- Do: produce the error tables.

**S3-15, S3-16 (team-wide)**

- Rehearsal: operate the AI-down fallback. Docs: write the dataset and labelling section.

## Janvi: Frontend lead (citizen and field)

Owns the citizen and field-team screens and the shared session pattern.

### Sprint 1

**S1-05 Auth UI on the real backend (owner; Tanishq supports)**

- Already in repo: /login (staff-sign-in.tsx already does the real password then TOTP flow), /signup, AuthModal.tsx, authApi.ts (it calls /api/auth/login, which does not exist, and falls back to localStorage), the BFF app/api/session/route.ts with lib/session.ts (httpOnly cookie drch_session), and proxy.ts, which only protects /dm. Also in repo (merged 16 to 18 Oct): services/citizenApi.ts, which sends the user identity in X-User-Id and X-User-Role headers to the SQLite endpoints under /api/citizen/*, and a citizen dashboard page at app/citizen/dashboard.
- Do: add a citizen proxy at app/api/citizen/[...path]/route.ts that mirrors app/api/dm: attach the bearer token from the cookie and reject cross-site writes. Let the login action accept a citizen sign-in (no MFA step in the response). Point the signup page at POST /auth/register. Extend the proxy.ts matcher to protect /report and /my-reports. Put the old localStorage fallback behind NEXT_PUBLIC_DEMO_MODE and keep it off on staging.
- **Owner (Janvi)** adds the citizen proxy at app/api/citizen/[...path]/route.ts, lets the login action accept a citizen sign-in, points signup at POST /auth/register, extends the proxy.ts matcher to /report and /my-reports, and puts the localStorage fallback behind NEXT_PUBLIC_DEMO_MODE. Also points citizenApi.ts at the citizen proxy and the new API, and stops sending X-User-Id and X-User-Role.
- **Support (Tanishq)** builds the shared layout and form components used by the auth pages and tests the pages.
- Needs from: Zalak register and login endpoints and the contract by Sprint 1 Day 2 (build against a stub until then); Siddh contract.
- Gives to: your S1-06 and Tanishq.
- Done when: register, login and a protected page work on the real database; a wrong password shows a generic message; localStorage is no longer used for auth. No browser request carries an identity header.

**S1-06 Citizen report form and My reports (owner; Tanishq supports)**

- Already in repo: CitizenReportModal.tsx (mock; writes to localStorage), CitizenDashboardView, and the Leaflet map with Nominatim search and reverse geocoding in page.tsx. The merged app/citizen/dashboard page can supply layout for My reports.

- Do: build a /report page: scenario (cyclone, flood, fire, earthquake), description, people affected, urgency, and location by GPS, by map pin clamped to Gujarat bounds, or by address search (debounced and cached; respect Nominatim's usage limits). Create an Idempotency-Key per form. Submit through the citizen proxy and show the reference on success. Build /my-reports with a list and a timeline that refreshes every 15 seconds.
- **Owner (Janvi)** builds the /report page (scenario, description, people affected, urgency, location by GPS, pin or address, Idempotency-Key, submit through the citizen proxy) and /my-reports with the 15-second timeline. Reuses the citizen dashboard layout where it fits and moves its data calls to the new /me/incidents API.
- **Support (Tanishq)** builds the shared layout and form components for the report and My reports pages and tests them with Gujarat inputs (inside, on the border, outside).
- Needs from: Zalak's report API; Nisarg's error codes (OUTSIDE_GUJARAT); Tanishq's shared Gujarat bounds constant.
- Done when: a UI submit creates a database row, a pin outside Gujarat is blocked in the browser and by the server, and a dispatcher action shows after a refresh.

**S1-08 Public map (support, owner Tanishq)**

- Do: contribute the shared Leaflet constants and map component (Gujarat bounds, tile settings) and review the How-to pages for the citizen and field flows.

## Sprint 2

**S2-04 Field-team dashboards (owner; Tanishq supports)**

- Already in repo: role-views/ServiceProviderDashboardView.tsx and RoleWorkspaceModal.tsx are mock responder views on localStorage; use them for layout ideas only.
- Do: build a /field route group (guard: role FIELD_TEAM; extend the proxy matcher; add an app/api/field proxy): assignment list; detail with incident summary, instructions and map pin; accept or reject (reason required); status buttons En route and On scene; an additional-team request form (capability, note); complete with a summary. Poll every 15 seconds. Mobile first.
- **Owner (Janvi)** builds the /field route group (role guard, proxy), assignment list and detail, accept or reject with reason, En route and On scene buttons, the additional-team request form and completion; polls every 15 seconds; mobile first.
- **Support (Tanishq)** provides the shared components and checks the screens on real devices.
- Needs from: Nisarg field endpoints with Swagger by Sprint 2 Day 1 and a seeded field user (CLI create-field); Dhruv seed-reset; Tanishq shared components.
- Done when: the seeded field user completes the whole flow at phone width and sees only their own team's dispatches.

**S2-11 Public alerts layer (owner; Rudra supports)**

- Do: add a map layer for GET /public/alerts with its own icons; items with is_fallback show a sample data badge; add a layer toggle.
- **Owner (Janvi)** adds the map layer for GET /public/alerts with its own icons, the sample-data badge for is_fallback items and a layer toggle; uses fixtures until the endpoint is live.

- **Support (Rudra)** builds GET /api/v1/public/alerts: official sources only, not expired, with source name and is_fallback.
- Needs from: Rudra's endpoint (use fixtures until it is live).

**S2-12 Donations (support, owner Shlok)**

- Do: build the campaign list, donate form and result page against Shlok's endpoints; share your session and form components.

**S2-13 Citizen information response (support, owner Zalak)**

- Do: add the response form in My reports for an incident in Needs information.

### Sprint 3

**S3-03 Frontend polish, accessibility and mobile (owner; Tanishq supports)**

- Do: loading, error and empty states; keyboard focus and contrast; phone-width layouts for the citizen and field screens; no console errors on the demo path. Fix demo-path pages first.
- **Owner (Janvi)** covers loading, error and empty states, keyboard focus, contrast, phone-width layouts and console errors on the citizen and field screens, demo path first.
- **Support (Tanishq)** does the same polish on the dispatcher and public screens.
- Done when: the core flows work with keyboard only and nothing breaks at phone width.

**S3-11 Payments hardening (support, owner Shlok)**

- Do: add UI states for declined, abandoned and timed-out payments.

**S3-15, S3-16, S3-17 (team-wide)**

- Rehearsal: play the citizen and the field user. Docs: write the frontend section. Defect buffer: own the frontend bugs in citizen and field screens (the largest share).

## Tanishq: Frontend (public site and dispatcher)

Owns the public Gujarat map and pages, and every change to the dispatcher workspace. Also available for GenAI work.

### Sprint 1

**S1-08 Public Gujarat map, About, usage instructions (owner; Janvi supports)**

- Already in repo: app/page.tsx has the Leaflet map (OSM and HOT tile options), an About modal, helplines and Nominatim; LiveOperationsLayer.tsx draws real incidents but only for a dispatcher session; the demo pins come from ResQContext.
- Do: make the home page public and data-driven: call GET /api/v1/public/incidents without login and draw verified incidents (colour by scenario or priority) with a district filter and a legend. Lock the map to Gujarat (setMaxBounds), keep the OSM attribution, and replace the Delhi and Odisha demo pins. Write the About page and a How-to-use page for the three flows: citizen (register, report, track), dispatcher (sign in with MFA, verify, assign), field (acknowledge, update, complete).

- **Owner (Tanishq)** makes the home page public and data-driven from GET /api/v1/public/incidents, adds the district filter and legend, locks the map to Gujarat, replaces the Delhi and Odisha pins and writes the About and How-to pages.
- **Support (Janvi)** contributes the shared Leaflet constants and map component (Gujarat bounds, tile settings) and reviews the How-to pages for the citizen and field flows.
- Needs from: Nisarg's public endpoint format (stub by Day 2); Siddh's district list; Janvi's shared map constants.
- Done when: the map shows seeded and live verified incidents, cannot be panned outside Gujarat, and the pages open without login.

**S1-09 Dispatcher console on real data; retire legacy mock (owner; Shlok, Zalak, Nisarg and Siddh support)**

- Already in repo: the 12 /dm pages and their panels already work against the real API.
- Do: run the flow on citizen-created incidents (verify, reject, needs information, duplicate, priority) and fix UI problems. Hide the legacy mock views (DispatchModal, RoleWorkspaceModal, SimulationBar, AlertsDrawer, the mock role dashboards) behind NEXT_PUBLIC_DEMO_MODE. Make sure no ResQContext data shows on staging.
- **Owner (Tanishq)** runs verify, reject, needs information, duplicate and priority on citizen-created incidents, fixes UI problems, hides the legacy mock views behind NEXT_PUBLIC_DEMO_MODE, makes sure no ResQContext data shows on staging, and keeps the shared defect list.
- **Support (Shlok and Zalak)** fix the backend defects on that list: Shlok those in the /dm routes, Zalak those in citizen incident code (for example reporter assumptions in incidents.py). **Support (Siddh)** reviews and merges the fixes and keeps the tests green.
- **Support (Nisarg)** runs the dispatch flow on citizen incidents (create, acknowledge as staff, escalation) and adds the defects he finds to Tanishq's list.
- Needs from: Zalak's report API merged; Shlok and Zalak fix the backend defects you report.
- Done when: a citizen report appears in the /dm queue, verify puts it on the public map, reject keeps it off, and no mock data is in the staging build.

**S1-05 and S1-06 (support, owner Janvi)**

- Do: build the shared layout and form components; test the pages with Gujarat inputs.

## Sprint 2

**S2-03 Dispatcher UI: recommendations, assignment, requests, resolution (owner; Nisarg supports)**

- Already in repo: dm/incidents/[id]/ has dispatch-panel.tsx, decision-panel.tsx, ai-panel.tsx and duplicates-panel.tsx.
- Do: add a team-recommendation panel (ranked teams with reasons and a clear line that it needs the dispatcher's confirmation) and a confirm-assign action that calls the existing dispatch creation with an Idempotency-Key and the chosen teams. Add a /dm/team-requests page to approve or decline additional-team requests. Add a resolution-confirm control when a field team reports completion.

- **Owner (Tanishq)** builds the team-recommendation panel, the confirm-assign action (Idempotency-Key, chosen teams), the /dm/team-requests page and the resolution-confirm control.
- **Support (Nisarg)** answers API questions and fixes contract problems the same day, and provides Swagger examples.
- Needs from: Nisarg endpoints by Sprint 2 Day 2; Siddh decision endpoints.
- Done when: a dispatcher can assign a recommended team in three clicks, and a user without the DISPATCH permission sees no assign action.

**S2-04 Field dashboards (support, owner Janvi)**

- Do: provide shared components and check the screens on real devices.

**S2-06 AI integration (support, owner Neeti)**

- Do: change ai-panel.tsx to show AI suggestion, not verified, plus confidence, factors, an out-of-scope message and the unavailable state.

**S2-10 Signals queue (support, owner Shlok)**

- Do: build /dm/signals with promote and dismiss and the AI pre-label.

### Sprint 3

**S3-04 About, usage and Gujarat safety content final (owner; Nisarg supports)**

- Do: finalise the About and How-to text so it matches the real screens. Load the Gujarat safety content through the existing DM content pages with a source for each; Nisarg checks the guidance against official pages.
- **Owner (Tanishq)** finalises the About and How-to text to match the real screens and loads the Gujarat safety content through the existing DM content pages with a source for each.
- **Support (Nisarg)** checks the cyclone, flood, earthquake and fire guidance against official pages and adds the source for each.
- Done when: every safety page names its source and the instructions match the UI.

**S3-03, S3-06, S3-08 (support)**

- S3-03: polish alongside Janvi. S3-06: draw the chart of accepted, modified and rejected AI suggestions from Zalak's endpoint. S3-08: build the photo-analyse control for Dhruvil.

**S3-15, S3-16, S3-17 (team-wide)**

- Rehearsal: play the dispatcher. Docs: write the dispatcher and public UI guide. Defect buffer: own dispatcher and public map bugs.

## Dhruv: QA and test lead

Owns the QA side: the test harness and fixtures, smoke and end-to-end tests, the scenario test, the role-matrix script and defect triage. CI/CD belongs to Siddh. Never holds secret values: Siddh enters them in the consoles and CI.

## Sprint 1

**S1-17 DB test harness, smoke script, seed reset (owner; Siddh supports for the smoke-account secret and CI wiring)**

- Already in repo: backend/tests/conftest.py, which skips DB tests when TEST_DATABASE_URL is not set; the CI job in ci.yml already provides a postgis/postgis:16-3.4 service and sets TEST_DATABASE_URL; a root-level test_citizen_api.py that CI does not run, because it only runs tests/.
- Do: add a pytest summary that prints how many tests ran and how many were skipped, and a guard that fails the run if no DB test ran (Siddh switches it on in CI). Put shared fixtures in conftest (drch_test database, citizen and staff users, later field users). Write scripts/smoke.sh: register a citizen, report, verify as a dispatcher, check the public API; for the dispatcher step use a staff account made with the CLI whose TOTP secret the script reads from an environment variable (agree the name with Siddh). Add make seed-reset.
- **Owner (Dhruv)** writes the ran-and-skipped summary and the guard, the shared fixtures, scripts/smoke.sh (reading the staff TOTP secret from an environment variable) and make seed-reset. He does not edit the CI workflow.
- **Support (Siddh)** creates the staff account for the smoke script with the CLI, keeps its TOTP secret and stores it as a CI and staging secret, tells Dhruv only the variable name, and wires the guard and the smoke step into CI (S1-18).
- Needs from: Siddh migrations; Zalak and Nisarg endpoints; Siddh for the test-account approach and the variable name.
- Gives to: Zalak and Nisarg (fixtures by Day 2); Siddh (guard and smoke script for S1-18 and S1-19).
- Done when: the test run prints the ran and skipped counts and fails if no DB test ran, the smoke script passes against compose, and make seed-reset restores the seed.

**S1-18 CI pipeline and staging deploy (support, owner Siddh)**

- Do: run the smoke script against staging when it is up and report the result; check that the README staging URLs and cold-start note are accurate; confirm the names-only variable checklist matches what the services read.
- Needs from: Siddh's staging URL.

**S1-01, S1-03, S1-19 (support)**

- S1-01: draft the README opening (Gujarat only, what is real, what is simulated) and give Siddh a checklist of the environment variable names each service needs (names only, never values).
- S1-03: provide the DB test fixtures to Zalak and Nisarg by Day 2.
- S1-19: run the Day 3 demo checklist and the smoke script and report pass or fail with the failing step.

## Sprint 2

**S2-14 Backend test hardening (owner; Zalak and Nisarg support)**

- Do: write DB-backed tests for field, signals, donations and public routes, and contract tests against the OpenAPI document.
- **Owner (Dhruv)** sets up the shared test setup and writes DB-backed tests for the field, signals and donations routes; makes sure at least 40 new tests run in CI with none skipped.
- **Support (Zalak)** writes DB-backed tests for the citizen, public and duplicate routes and the contract tests against the OpenAPI document.
- **Support (Nisarg)** writes the cross-team isolation tests for the field router.
- Needs from: the routers from Zalak, Nisarg and Shlok.
- Done when: at least 40 new tests run in CI with none skipped.

**S2-16 Staging redeploy and ingestion cron (support, owner Siddh)**

- Do: extend the smoke script to the field flow and a donation, and check that the staging webhook URL answers.
- Needs from: Siddh's staging; the field and donation endpoints.

**S2-01, S2-12, S2-15 (support)**

- S2-01: set up the test fixtures for field users.
- S2-12: keep the payment tests in the CI run (Siddh owns the workflow).
- S2-15: run the gate checklist and the smoke script and report pass or fail.

## Sprint 3

**S3-01 End-to-end scenario tests (owner; Zalak and Nisarg support)**

- Do: automate the cyclone scenario from citizen report to closure (API level, browser level if time allows) and run it three times in a row on staging.
- **Owner (Dhruv)** automates the cyclone scenario from citizen report to closure and runs it three times in a row on staging.
- **Support (Zalak)** writes the API-level steps for the citizen and public parts.
- **Support (Nisarg)** writes the API steps for assignment and the field flow.
- Needs from: Zalak and Nisarg write their steps; all routers stable at Sprint 3 Day 1.

**S3-17 Defect triage (owner; Siddh approves release blockers)**

- Do: triage defects into P0, P1 and P2 and assign each to its module owner; keep the list current through Sprint 3 and report the P0 count at each rehearsal.
- **Owner (Dhruv)** keeps the defect list, assigns each defect to its module owner and re-tests the fixes.
- **Support (Siddh)** approves the P0 list and decides what blocks the release.
- Done when: no P0 is open at the release tag and every P1 is fixed or has a written decision.

**S3-02 Security review (support, owner Siddh)**

- Do: build the role-matrix script (public, citizen, field, dispatcher, admin against every route group, expected status codes supplied by Zalak, Nisarg and Shlok), run it and hand the results to Siddh.

- Needs from: all routers stable at Sprint 3 Day 1; the expected status codes from the route owners.

**S3-12 Production-like deployment (support, owner Siddh)**

- Do: deploy from the docs on a clean machine with dummy values (Dhruv did not write them), time it and report every step that did not work.

**S3-15 Demo script (support, owner Siddh)**

- Do: draft the demo script from the scenario test, with who plays which role; run the infrastructure checklist in the rehearsals.

## Handoffs that block other people

If one of these is late, say so the same day.

| From | To | What | Needed by |
|---|---|---|---|
| Neeti | Dhruvil, Rudra | Labelling guideline and template | Sprint 1 Day 1 |
| Siddh | Zalak, Nisarg, Shlok, Rudra, Neeti | Contract template for their sections | Sprint 1 Day 1 |
| Nisarg | Siddh | Gujarat agencies and teams seed rows with capability keys | Sprint 1 Day 1 |
| Siddh | Everyone | Contract doc, owners signed off | Sprint 1 Day 2 |
| Siddh | Zalak, Nisarg, Shlok, Rudra | Migrations 03 and 04 merged | Sprint 1 Day 2 |
| Siddh | Zalak, Nisarg | CI job that runs DB tests and prints the ran and skipped counts | Sprint 1 Day 2 |
| Dhruv | Zalak, Nisarg | DB test fixtures | Sprint 1 Day 2 |
| Siddh | Dhruv | Name of the smoke-account environment variable (value stays with Siddh) | Sprint 1 Day 2 |
| Nisarg | Zalak | geo.py (bounding-box stub is enough) | Sprint 1 Day 2 |
| Zalak | Janvi | Register and login endpoints | Sprint 1 Day 2 |
| Nisarg | Tanishq | Public incidents response format | Sprint 1 Day 2 |
| Neeti | Rudra | rules.py (keyword rules port) for the service skeleton | Sprint 1 Day 2 |
| Dhruvil | Neeti | Image weights found or not | Sprint 1 Day 2 |

| From | To | What | Needed by |
|---|---|---|---|
| Zalak | Janvi | Citizen report API | Sprint 1 Day 3 |
| Siddh | Rudra | Render reachability result per source | Sprint 1 Day 3 |
| Shlok | Siddh | Campaigns and demo-incident seed rows | Sprint 1 Day 3 |
| Rudra, Dhruvil | Neeti | Labelled texts (Neeti labels her own share) | Sprint 1 Day 4 |
| Shlok | Nisarg | Shared dispatch-transition refactor merged | Sprint 2 Day 1 |
| Nisarg | Janvi | Field endpoints and a seeded field user | Sprint 2 Day 1 |
| Neeti | Zalak | Allowed labels and /triage contract for the LLM adapter | Sprint 2 Day 1 |
| Dhruvil | Zalak | Text-similarity module (default weights) and labelled duplicate pairs (CSV) | Sprint 2 Day 2 |
| Rudra | Shlok | Signals table filled with sample rows | Sprint 2 Day 2 |
| Nisarg | Tanishq | Team recommendation endpoints | Sprint 2 Day 2 |
| Shlok | Janvi | Donation endpoints | Sprint 2 Day 2 |
| Siddh | Shlok | Gateway and webhook secrets entered on staging (values stay with Siddh) | Sprint 2 Day 2 |
| Neeti | Siddh, Dhruvil | Model artifact and final /triage contract | Sprint 2 Day 3 |
| Dhruvil | Neeti | Tuned similarity weights and threshold for sign-off | Sprint 2 Day 4 |
| Dhruv | Siddh | Extended smoke script (field flow and donation) | Sprint 2 Day 4 |
| Siddh | Dhruv | All routers stable for the scenario test; staging URL | Sprint 3 Day 1 |
| Every member | Zalak | Their section of the final report | Sprint 3 Day 4 |